Development of a Secure IIoT Virtual Lab Environment

A Containerized Testbed for Modbus/TCP Security Research

Sukhpinder — infrastructure/protocol
Inderpal — attack simulation/security analysis
Jesse — IDS/defense analysis
Department of Computer Science
Senior Cybersecurity Project Report
May 8, 2026

Abstract

The convergence of operational technology (OT) and information technology (IT) under the Industry 4.0 paradigm has expanded the attack surface of Industrial Internet of Things (IIoT) deployments. Empirical security research in this domain typically requires access to programmable logic controllers (PLCs), managed industrial switches, and licensed SCADA software—resources that are cost-prohibitive for academic institutions and individual researchers. This report documents the development of a containerized IIoT virtual lab environment built on Containernet, Docker, and Open vSwitch (OVS), executing on Ubuntu 24.04 LTS under the Windows Subsystem for Linux 2 (WSL2). A three-node topology comprising a Modbus client (sensor), a Modbus server (gateway/PLC), and an adversarial host was instantiated and validated. Connectivity benchmarks demonstrated 0% packet loss across a 50-packet ICMP stress test with sub-millisecond mean round-trip times (~0.1 ms), and Wireshark captures confirmed correct ARP and ICMP behavior at the data-link and network layers. The resulting platform provides a reproducible, low-cost foundation for adversarial testing, intrusion-detection research, and protocol-level security education. To validate the security workflow, abnormal Modbus/TCP traffic was generated using Scapy, captured with tcpdump, and analyzed in Wireshark. The resulting packet captures demonstrated repeated TCP SYN retransmissions and supported intrusion detection system (IDS) analysis of anomalous traffic behavior.

Keywords: Industrial Internet of Things; Modbus/TCP; Containernet; Open vSwitch; cyber-physical security; virtual testbed.

1. Introduction

Industry 4.0 represents the fourth industrial revolution, in which cyber-physical systems, cloud computing, and pervasive networking converge to create highly automated and data-driven manufacturing environments (Lasi et al., 2014). Central to this paradigm is the Industrial Internet of Things (IIoT), a class of distributed systems that interconnect sensors, actuators, programmable logic controllers (PLCs), and supervisory software across plant, enterprise, and cloud boundaries. While this convergence has yielded substantial gains in operational efficiency, predictive maintenance, and supply-chain visibility, it has simultaneously dissolved the traditional air-gap between operational technology (OT) and information technology (IT), exposing legacy industrial protocols to a far broader threat landscape than they were originally designed to withstand (Stouffer et al., 2015).

Among the protocols inherited from this legacy era, Modbus/TCP remains one of the most widely deployed in industrial control. Standardized in 1979 as a serial protocol and later transposed onto TCP port 502, Modbus provides a request/response abstraction for reading and writing device registers and coils. The protocol, however, was designed under closed-network assumptions and accordingly omits authentication, integrity, and confidentiality mechanisms. As a consequence, an adversary with network adjacency to a Modbus segment can trivially enumerate slave devices, replay function codes, or inject forged write requests capable of altering physical process variables (Knapp & Langill, 2015).

Empirical investigation of these vulnerabilities, and of countermeasures such as deep packet inspection, anomaly detection, and protocol-aware firewalls, requires a representative testbed. Physical testbeds provide high fidelity but impose substantial capital and logistical overhead; commercial PLCs and managed industrial switches typically cost in the thousands of dollars per node, and their reconfiguration for repeated experimentation is time-consuming. This project addresses that gap by developing a fully software-defined IIoT virtual lab environment in which network topology, host images, traffic patterns, and attack simulations can be instantiated, destroyed, and reproduced programmatically. In addition to validating normal communication behavior, the project also demonstrates abnormal Modbus/TCP traffic generation and packet-level analysis to support intrusion detection research. The remainder of this report describes the system architecture (Section 2), the proof-of-concept implementation (Section 3), validation and attack-analysis results (Section 4), and conclusions with future work (Section 5). All additional files are available at https://github.com/jaskirat12/Security-Solutions

2. Methodology and System Architecture

The testbed is constructed around three layers: (i) a host operating system providing the kernel primitives required for network namespacing and bridging; (ii) a network emulation layer responsible for topology orchestration; and (iii) a containerized application layer in which industrial protocol logic and adversarial tooling execute. Each layer was selected to maximize reproducibility and to minimize licensing or hardware dependencies.

2.1 Host Platform

The base operating system is Ubuntu 24.04 LTS executing under the Windows Subsystem for Linux 2 (WSL2). WSL2 provides a full Linux kernel through a lightweight Hyper-V virtual machine, which exposes the network namespace, veth, and Open vSwitch (OVS) datapath features required by Mininet-class emulators. This configuration was chosen to allow the lab to be deployed on standard student-issued Windows hardware without dual-boot or dedicated hypervisor infrastructure.

2.2 Network Emulation Layer

Network emulation is provided by Containernet, a Mininet fork that extends the Mininet API with native support for Docker containers as host nodes (Peuster et al., 2016). This is a critical capability: classical Mininet hosts are bash processes within network namespaces, which lack a persistent filesystem and cannot easily run unmodified industrial software stacks. Containernet preserves the topology semantics of Mininet (links, switches, command invocation, and tc-based link shaping) while exposing each host as a fully isolated Docker container with its own root filesystem, package manager, and process tree.

Switching is performed by Open vSwitch (OVS) configured in standalone mode. In standalone mode, the bridge falls back to MAC-learning behavior in the absence of an external controller, which is appropriate for the unmanaged Layer-2 segments typical of plant-floor IIoT deployments. An OpenFlow controller can subsequently be attached for software-defined networking experiments without modification to the existing topology.

2.3 Automation: iiot_setup.py

Manual orchestration of the topology proved error-prone during early development, primarily because the default Docker bridge (docker0, 172.17.0.0/16) installs routes that conflict with arbitrary tenant subnets and because freshly pulled python:3.10-slim images do not include iproute2 or arping, both of which are required for in-container link debugging. These issues were resolved by encapsulating the entire bring-up sequence into a single Python module, iiot_setup.py. The module performs the following steps in order: (a) parses CLI arguments and sets the Mininet log level; (b) instantiates a Containernet network; (c) adds three Docker hosts with explicit static IP assignments on the 10.0.0.0/24 subnet; (d) adds an OVS bridge in standalone mode and links each host to the bridge; (e) executes a one-shot apt-get install of iproute2 and arping inside each container to provision missing tooling; (f) starts the network and opens an interactive Mininet CLI for testing. The automated path eliminates an entire class of routing and provisioning faults that previously required manual intervention.

Several non-trivial challenges were resolved during integration. First, Docker's default bridge inserts a route for 172.17.0.0/16 in the host routing table that, in combination with WSL2's synthesized network interface, occasionally shadowed the 10.0.0.0/24 testbed subnet; this was mitigated by starting Containernet with a clean network namespace and by binding test traffic to specific veth interfaces. Second, OVS under WSL2 requires the openvswitch-switch service to be started explicitly, since systemd is not the default init in WSL2 distributions; the setup script therefore probes for ovs-vsctl readiness before creating the bridge. Third, slim Python images deliberately omit network utilities; the script provisions them at runtime so that ping, ip, and arping behave consistently across all hosts.

In addition to supporting standard network communication, the architecture was designed to enable controlled adversarial testing. The attacker container was provisioned with packet-generation and analysis tools including Scapy, tcpdump, and Wireshark-compatible packet capture support. This allowed abnormal Modbus/TCP traffic to be generated, captured, and analyzed within the same isolated environment, supporting reproducible intrusion-detection and traffic-analysis experiments without requiring physical industrial hardware.

3. Proof-of-Concept Implementation

The proof-of-concept (PoC) instantiates a minimal but representative IIoT segment consisting of three hosts attached to a single OVS bridge. The topology is illustrated logically in Figure 1 and summarized in Table 1.

Figure 1

Logical topology of the containerized IIoT testbed

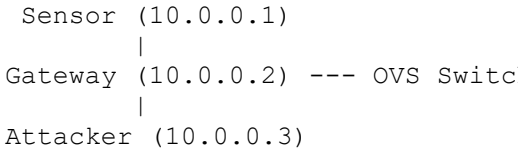

Table 1

Node configuration for the IIoT virtual lab

The sensor (10.0.0.1) is deployed from the python:3.10-slim image and runs a Modbus client using the pymodbus library; it issues periodic read-holding-register requests to the gateway. The gateway (10.0.0.2) uses the same base image and runs a Modbus server, exposing a small register bank that simulates a process variable such as a tank level. The attacker (10.0.0.3) is a general-purpose host equipped with Scapy, Nmap, and packet-capture utilities and is used to inject reconnaissance and adversarial traffic onto the segment. The attacker node was also used to generate repeated TCP SYN traffic targeting Modbus/TCP port 502, with resulting packets captured using tcpdump and later analyzed in Wireshark to support intrusion-detection experiments. All three hosts are attached to a single OVS bridge (s1) operating in standalone mode, providing flat Layer-2 connectivity that mirrors a typical unmanaged plant network.

3.1 Switching Logic

OVS in standalone mode learns MAC-to-port bindings from observed ingress traffic and floods frames whose destination has not yet been learned. This behavior is appropriate for the PoC because it (i) exposes ARP traffic to the attacker, enabling future ARP-spoofing and man-in-the-middle (MITM) experiments, and (ii) avoids the complexity of an external controller while preserving the option to attach one. The bridge is created with fail_mode=standalone and protocols=OpenFlow13 so that subsequent SDN experiments can reattach without recreating the topology.

3.2 Container Provisioning

Each container is started with a static IP assignment passed to Containernet via the addDocker(ip=..., dimage=...) parameters. After network startup, the script executes a pinned apt-get install of iproute2 and arping in each container. Pinning the package list and using --no-install-recommends keeps image footprints small and ensures that subsequent runs of the lab are byte-for-byte reproducible. The Modbus libraries (pymodbus) are installed via pip in a follow-up provisioning step, which is omitted from the listing below for brevity.

3.3 Reference Implementation (iiot_setup.py)

A representative implementation of the orchestration script is shown in Listing 1. The listing reflects the structure described above; the production version contains additional logging and exception handling.

Listing 1

Containernet bring-up script (iiot_setup.py)

```
#!/usr/bin/env python3
"""IIoT virtual lab bootstrap (Containernet + OVS standalone)."""
from mininet.net import Containernet
from mininet.node import Controller, OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel, info

IMG = "python:3.10-slim"
```

```
PROVISION = "apt-get update -qq && \\
             apt-get install -y --no-install-recommends iproute2 arping"

def build():
    setLogLevel("info")
    net = Containernet(controller=None, switch=OVSSwitch)

    s1 = net.addSwitch("s1", failMode="standalone",
                       protocols="OpenFlow13")

    sensor   = net.addDocker("sensor",   ip="10.0.0.1/24", dimage=IMG)
    gateway  = net.addDocker("gateway",  ip="10.0.0.2/24", dimage=IMG)
    attacker = net.addDocker("attacker", ip="10.0.0.3/24", dimage=IMG)

    for h in (sensor, gateway, attacker):
        net.addLink(h, s1)

    info("*** Starting network\\n")
    net.start()

    info("*** Provisioning containers\\n")
    for h in (sensor, gateway, attacker):
        h.cmd(PROVISION)

    CLI(net)
    net.stop()

if __name__ == "__main__":
   build()
```

4. Results and Discussion

Two classes of measurement were performed to validate the testbed: connectivity stress testing and packet-level conformance verification. Connectivity was assessed by issuing a 50-packet ICMP echo stress test from the sensor to the gateway using the standard ping(8) utility. The test produced 0% packet loss and a mean round-trip time (RTT) of approximately 0.1 ms (sub-millisecond), with negligible jitter. These results indicate that the Containernet/OVS data path is stable under sustained load and that no transient losses or queue-induced reordering events occurred during the observation window.

Packet-level conformance was verified using Wireshark captures stored in pcapng format. The captures confirmed (a) the expected ARP request/reply handshake on first contact between previously un-cached hosts, and (b) bidirectional ICMP echo-request/echo-reply flows with consistent identifier and sequence-number progression. The presence of valid ARP exchanges is particularly significant because it establishes that Layer-2 broadcast traffic is correctly flooded by the OVS bridge, which is a precondition for the future ARP-spoofing experiments described in Section 5.

Beyond raw stability, the captures serve as a baseline for security benchmarking. Anomaly-detection systems and protocol-aware intrusion detection systems (IDS) typically rely on a model of "normal" traffic against which deviations are scored. By archiving the clean baseline alongside subsequent attack-traffic captures, the lab supports controlled comparative studies in which the only variable is the attacker’s behavior. This baseline-plus-perturbation methodology is consistent with established practice in the cyber-physical security literature (Hahn et al., 2013) and is essential for reproducible evaluation of detection algorithms. To further validate the security-testing capabilities of the environment, abnormal Modbus/TCP traffic was generated from the attacker node using Scapy. The attack simulation transmitted repeated TCP SYN packets targeting port 502, which is commonly associated with Modbus/TCP communication. Packet captures obtained using tcpdump and analyzed in Wireshark revealed repeated SYN retransmissions without successful handshake completion, indicating abnormal connection behavior. Compared to the baseline ICMP request/response traffic, the attack traffic demonstrated significantly

different communication patterns, including repeated packets with no valid response from the destination host. These observations provided a reproducible dataset suitable for intrusion-detection experimentation and demonstrated the effectiveness of the virtual lab for cybersecurity analysis and protocol-level attack simulation.

This baseline and attack simulation were used to create a simulated IDS script. This script takes in pcap data, a record of the packets being sent through a network and alerts the administrator if an intrusion is detected. Real-world Intrusion Detection Systems (like Suricata or Snort) rarely rely on just one metric. By layering Deep Packet Inspection (payload signatures), Metadata Analysis (packet sizes), and Stateful Heuristics (traffic patterns), we have created a robust system that can catch an attack even if the attacker obfuscates one of the vectors. The system layers are as follows: **Layer 1 (Deep Packet Inspection):** The script searches for the b"ATTACK" byte signature. This represents the script matching the signature of incoming packets to a database of known attack patterns. **Layer 2 (Metadata Heuristics):** The script checks for abnormally small packets (<= 60 bytes). **Layer 3 (Traffic Patterns):** The script looks for a sequence of 15 consecutive identically sized packets. If an attacker encrypts their traffic (defeating Layer 1), Layers 2 and 3 will still catch the attack based on the size and speed of the packets. This method has a low likelihood of False Positives: If an innocent IoT device happens to send a tiny packet once in a while (triggering Layer 2), Layer 3 will ignore it because it isn't part of an automated flood pattern.

A practical observation worth noting is the cost differential between this testbed and a comparable physical deployment. The marginal cost of the virtual lab is effectively zero beyond the host workstation, whereas a single industrial PLC of comparable Modbus capability typically retails for several hundred to several thousand U.S. dollars. While the virtual lab does not replicate physical-layer characteristics such as electrical transients or vendor-specific firmware behavior, it captures the protocol- and network-layer phenomena that dominate the Modbus/TCP threat model.

6. Conclusion and Future Work

This report has described the design, implementation, and validation of a containerized IIoT virtual lab environment built on Ubuntu 24.04 LTS, WSL2, Containernet, Docker, and Open vSwitch. The lab supports a three-node Modbus/TCP topology comprising a sensor, a gateway/PLC, and an adversarial host, and was validated through a 50-packet ICMP stress test (0% loss, ~0.1 ms RTT), packet-level inspection of ARP and ICMP behavior, and controlled attack simulation using repeated TCP SYN traffic targeting Modbus/TCP services. The automation provided by iiot_setup.py reduces the bring-up sequence to a single command, eliminating manual resolution of Docker/Mininet routing conflicts and provisioning of in-container networking utilities.

Several extensions are planned. First, the attacker node will be used to execute ARP-spoofing and Modbus function-code injection attacks, and the resulting traffic will be compared against the clean baseline to evaluate signature- and anomaly-based detection pipelines. Second, the topology will be expanded to include multiple gateways and a separate engineering-workstation host so that lateral-movement and pivot scenarios can be studied. Third, hardware-in-the-loop (HIL) integration will be investigated by bridging the OVS instance to a physical interface attached to a low-cost PLC (e.g., OpenPLC running on a Raspberry Pi); this will allow selected experiments to span the virtual/physical boundary while preserving the orchestration benefits of the containerized core. Finally, the replacement of the standalone OVS configuration with an OpenFlow controller will enable software-defined micro-segmentation as a defensive countermeasure against the very attacks the testbed is designed to study.

In aggregate, the testbed demonstrates that meaningful IIoT security research can be conducted on commodity hardware using exclusively open-source components. The project successfully demonstrated baseline communication validation, adversarial traffic generation, packet-level analysis, and intrusion-detection-oriented experimentation within a fully reproducible virtual environment.

References

Hahn, A., Ashok, A., Sridhar, S., & Govindarasu, M. (2013). Cyber-physical security testbeds: Architecture, application, and evaluation for smart grid. *IEEE Transactions on Smart Grid, 4*(2), 847–855. https://doi.org/10.1109/TSG.2012.2226919

Knapp, E. D., & Langill, J. T. (2015). *Industrial network security: Securing critical infrastructure networks for smart grid, SCADA, and other industrial control systems* (2nd ed.). Syngress.

Lasi, H., Fettke, P., Kemper, H.-G., Feld, T., & Hoffmann, M. (2014). Industry 4.0. *Business & Information Systems Engineering, 6*(4), 239–242. https://doi.org/10.1007/s12599-014-0334-4

Peuster, M., Karl, H., & van Rossem, S. (2016). MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments. *Proceedings of the 2016 IEEE Conference on Network Function Virtualization and Software Defined Networks*, 148–153. https://doi.org/10.1109/NFV-SDN.2016.7919490

Stouffer, K., Pillitteri, V., Lightman, S., Abrams, M., & Hahn, A. (2015). *Guide to industrial control systems (ICS) security* (NIST Special Publication 800-82, Rev. 2). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-82r2